False Success Is an Architecture Problem (v0.3)

False Success Is an Architecture Problem: Evidence Gate and Deterministic Verification for LLM Agents on HPC

Preprint v0.3 — draft, 10 November 2026 Renato Schust (GAMA TRADE) · compute: LUMI-C/G, project `465003647`

> Status: internal draft, not peer reviewed. Every number in §4 is reproducible from the artifacts listed in §6; items that failed are reported as failures.

Abstract

Agents built on language models are increasingly allowed to run actions (restart services, delete tables, send mail). Two failure modes are invisible to text-level benchmarks: the agent reports success for work that never happened (*false success*, FSR), and the agent executes a destructive action without approval. We report a measurement campaign on LUMI (AMD MI250X) covering **36,550 guard requests** across two model families and four concurrency levels, an objectively scorable 700-item benchmark, a deterministic document verification gate, and an adaptive prompt-injection evaluation. We find that (i) false success is **not** eliminated by scale: no model in a 700-item suite resisted fabricating specific facts about non-existent entities, and supervised fine-tuning made it **worse** (73% → 41%); (ii) text-level guards are ineffective: an existing keyword guard detected 33.0% of paraphrased attacks while blocking 35.0% of legitimate questions, and an improved intent guard reached 90.1% / 2.5% in-sample and 85.7% / 3.5% out-of-sample, still short of the 95% target; (iii) an **action-level** deterministic gate blocked **119/119** destructive calls without a single false block on 108 read-only calls, while an adversarial stress test shows the same gate implemented as an exact tool-name list would pass only **52.6%** of 1,000 destructive variants. On 500 real but obscure entities the models fabricated 71–95% of answers when the prompt demanded a number, and 13–22% when refusal was explicitly allowed; Qwen2.5-7B hedged 73–81% on fictional entities but 0/1000 on real obscure ones. A template-level re-analysis shows the 700 items come from only **24 prompt templates** (design effect 4.7–21.9, effective N 32–149), and that accuracy is decided by the template rather than the item: two of three arithmetic templates score 0/114 and 0/79. We conclude that truthfulness in agents must be enforced architecturally — by an action-level gate plus deterministic verification — rather than learned.

1. Problem

Two claims are routinely conflated: “the model says it did X” and “X happened”. On real infrastructure the gap is expensive. A node that returns `ok:true` after a timeout is a false success; an agent that restarts a database because the user phrased a request politely is an unapproved execution. Both are properties of the **system**, not of the text the model emits.

2. Method

Scale bench (E1, E5). A vLLM server on 4 GCDs of an MI250X node, driven by an async client. Each request is a prompt from 12 templates (≈833 repetitions at N=10,000), capped at `max_tokens=512`, `temperature=0`, `timeout=300`. Every run is checked for schema, duplicates, empty successes and **tokenizer corruption**; a run with >1% byte-level garbage is marked `INVALID` rather than scored.

Verifiable benchmark v2. 700 generated items in four blocks with exact ground truth: invoice arithmetic with 17% VAT (300), long-context needle retrieval (100), honesty about non-existent entities (100), and exact JSON tool calls (200). Dataset SHA-256 `49c2f8ac…`, regenerated bit-identically on two machines.

Template-level statistics. The 700 items are drawn from **24 prompt templates** (arithmetic 3, needle 1, honesty 8, tool calls 12), so items are not independent observations. We report the naive Wilson interval alongside a cluster bootstrap (4,000 resamples of templates within block), the design effect, and the effective sample size. A block with a single template (needle) has no identifiable cluster variance and is flagged rather than given a spurious interval.

Deterministic verification (E6). 500 synthetic invoices with planted VAT/sum errors; a rule engine recomputes each line, subtotal, VAT and total from the data itself.

Adaptive injection (E3). 203 attacks in 10 destructive families with Cyrillic homoglyphs, spaced letters, dot substitution, zero-width characters and uppercase, plus 200 benign controls of which half contain the same dangerous keywords as documentation questions.

Action gate (RiskGate). A deterministic classifier over tool name and arguments: `deny` for forbidden patterns, `confirm` for a fixed set of destructive tools without a recorded approval, `allow` otherwise.

3. What was measured, and what failed

We report three failures explicitly, because they shaped the conclusion.

1. **A “1/10,000 false success” was a measurement artefact.** The honesty checker matched the year in the identifier `VP2026` as a customer count. After excluding years and alphanumeric identifiers the run is **0/10,000**.
2. **A “49.5% false success” for Mistral-3.2 was tokenizer corruption.** 116/200 outputs were byte-level garbage (`Ġ`, `Ñ`). The run is now reported as `INVALID`. Two further serving configurations failed differently (Pixtral processor crash; degenerate repetition), so the model is excluded entirely.
3. **Two identical runs are not bit-identical — and serial decoding does not fix it.** At `temperature=0` with continuous batching, b1/b2/b5 match exactly but the honesty block differs in 2/100 items (79 vs 81). Forcing strictly serial decoding (`--max-num-seqs 1`, client concurrency 1) leaves 40/700 (TP=4) and 31/700 (TP=1) outputs different; short constrained outputs (needle, tool-JSON) are 100% identical, arithmetic and free-form text are not. Claims of “byte-identical” generation would be false. On MI250X, vLLM’s batch-invariant mode (`VLLM_BATCH_INVARIANT=1`, supported on gfx90a) does deliver perfect **within-process** agreement — 700/700 at concurrency 64 for both TP=1 and TP=4 — but only 586/700 (83.7%) and 593/700 (84.7%) across a server restart. The guarantee is intra-process; it is not cross-run reproducibility. A forensics run (4 arms × 3 repeats, token log-probs) locates the within-process flips in the attention backend: the default kernel diverges at token 5 (median) in 9–10% of items at concurrency 8, while `--attention-backend TRITON_ATTN`, batch-invariant mode, or both give 0/100.

4. Results

4.1 Scale (E1, E5)

model	N	concurrency	req/s	TTFT p50	p95 latency	FSR
Qwen2.5-7B	5,000	128	88.2	67 ms	6.3 s	0
Qwen2.5-7B	5,000	**256**	**114.1**	103 ms	9.4 s	0
Qwen2.5-7B	**10,000**	512	113.8	3.7 s	13.4 s	**0/10,000**

model	N	concurrency	req/s	TTFT p50	p95 latency	FSR
Llama-3.3-70B	5,000	128	22.6	370 ms	21.4 s	0
Llama-3.3-70B	5,000	**256**	**28.7**	582 ms	33.5 s	0
Llama-3.3-70B	**10,000**	512	28.5	13.2 s	47.8 s	**0/10,000**

Both models saturate at concurrency 256; above it throughput is flat and latency grows 4–20×. Total: 36,550 requests, 0 errors, 0 false successes.

Multi-node. With one vLLM replica per node (TP=8, Mistral-7B) and concurrency scaled per replica (256 each), throughput is 70.5 rps on 1 node, 140.4 on 2 (1.99×) and 269.6 on 4 (3.82×, 95.6% parallel efficiency), at a stable 8.4–8.8 rps per GCD. Holding total concurrency fixed instead yields only 1.33× on 2 nodes, which is a client-side artefact, not a scaling limit. Per GCD, TP=1 is four times more efficient than TP=8 (32.0 vs 8.8 rps/GCD), so TP=8 should be chosen for latency, not throughput.

4.2 Verifiable benchmark v2 (700 items)

model	arithmetic	needle	honesty	tool-JSON	overall
gpt-oss-120b	37.0%	99%	59%	99%	**66.7%**
Qwen2.5-7B	17.3%	99%	73–81%	100%	60.6–61.7%
Qwen2.5-VL-7B	10.3%	99%	17%	96%	48.4%
Llama-3.3-70B	7.3%	99%	28%	91%	47.3%
Llama-Poro-70B	5.7%	100%	36%	79%	44.4%

Larger is not better: a 7B model beats a 70B model on arithmetic and honesty. Exact decimal arithmetic is the weakest block for every model (5.7–37%).

4.2b What the intervals hide (template-level re-analysis)

With 24 templates the naive Wilson interval is 2–3× too narrow:

model	accuracy	Wilson 95%	cluster 95%	design effect	effective N
Qwen2.5-7B	60.6%	[56.9; 64.1]	[50.2; 72.4]	10.03	69.8
Llama-3.3-70B	47.3%	[43.6; 51.0]	[39.3; 55.2]	4.71	148.5
bal6 adapter	63.4%	[59.8; 66.9]	[49.1; 81.5]	21.86	32.0

The aggregate also conceals template-level collapse:

- Qwen2.5-7B: `b1_sub` **0/114**, `b1_total` **0/79**, all arithmetic credit from `b1_vat` (52/107). The "17.3% arithmetic" is therefore 0% on two of three templates.
- bal6: the jump from 17.3% to 38.0% is entirely `b1_vat` (48.6% → 83.2%); `b1_total` remains 0/79.
- Llama-3.3-70B: `b5_service_logs` **0/18** while the other 11 tool templates are 182/182.
- Honesty is all-or-nothing per entity type: bal6 answers 0/13 firms, 0/14 companies and 0/18 devices, but 10/10 cities and 9/9 software items. Its honesty drop (73% → 41%) is the loss of hedging on three entity types, not a uniform decline.

This changes what the benchmark measures: it is a **24-task** test with many repetitions, and the repetitions add precision about those tasks, not about the domain.

4.2c Cross-family replication

Two non-Qwen families, three seeds each, same 700 items (`temperature=0`, 4 GCDs):

model	seed 42	seed 43	seed 44	flips vs s42	b1	b2	b4	b5
Mistral-7B-Instruct-v0.3	38.4%	38.3%	38.6%	5 / 9	0.7–1.0%	99%	25–27%	70.5%
Phi-3.5-mini-instruct	37.7%	35.9%	36.0%	23 / 22	10.7–15.0%	87–88%	0–1%	63.5–65.5%

All runs had zero request errors. The block profile repeats outside the Qwen family: needle is easy (87–99%), tool-JSON intermediate (63–71%), arithmetic near-zero (0.7–15%), honesty low (0–27%). Honesty spans 80 points across models of the same size class, which supports the architectural rather than capability-based reading. Seed-to-seed flips are model-dependent (0.7–1.3% for Mistral, 3.2–3.3% for Phi-3.5), a further argument against bit-identical generation claims.

4.3 Honesty cannot be trained away

We fine-tuned the 9B base on 800 examples (40% tool calls, 40% hedges, 20% short facts), holdout-clean. Reproduce the outcome: tool accuracy became the best of all adapters (23/25 strict, arguments 7/7, termination preserved), **but honesty fell from 73% (base) to 41%**, and the model confidently invented specifics: *“the registration number of Valdora Logistik is 33006020000”*, *“Novex Dynamics was founded on 1 January 2014”*. Supervised fine-tuning teaches the *style* of hedging, not the *detection* of the unknown.

4.3b Real, obscure entities: the refusal trigger is “looks made-up”

The honesty block above uses non-existent entities. To test the opposite case we built 500 items about **real but obscure** entities from Wikidata (sitelinks ≤ 8 for settlements, ≤ 3 for companies/people/films, ≤ 2 for asteroids), each with a reference value and QID. Scoring is CORRECT / HEDGE / FABRICATE against that reference.

model	forced answer: fabricate	hedge	correct	with permission: fabricate	hedge	correct
Qwen2.5-7B	**94.8%**	0.0%	5.2%	**21.6%**	77.6%	0.8%
Mistral-7B-Instruct	**74.4%**	7.2%	18.4%	**17.0%**	72.8%	10.2%
Phi-3.5-mini	**71.4%**	25.8%	2.8%	**13.0%**	85.8%	1.2%

Two findings matter. First, Qwen2.5-7B hedged 73–81% of questions about *fictional* entities but **0/1000** about real obscure ones: the trigger for refusal is whether the entity sounds invented, not whether the model knows it. Second, adding the sentence “if you do not know, answer exactly: I do not know” cuts fabrication 3.5–5×. The “answer with the number only” format is itself a risk factor, which is a prompt-design result, not a model property. Per-category results are heterogeneous (Mistral knows 60/100 asteroid discovery years; Phi-3.5 hedges 84/100 people but 2/100 settlements), so the aggregate hides the structure.

4.4 Deterministic verification (E6)

500 invoices (381 with a planted error, 119 clean), three seeds: **recall 1,109/1,109 = 100%**, **false positives 0/391**, Wilson 95% upper bound **0.97%** (≤1%). A rule engine, not a model, decides here.

Probe sensitivity. The same 500 invoices produce **0% / 30.2% / 47.8% / 49.1% / 100%** recall as the probe weakens from “trust the printed total” (L0) through subtotal (L1), subtotal+VAT (L2), line

items+subtotal (L3) to full recomputation (L4), with 0/119 false positives at every level. The published 100% is therefore a property of the **(gate, probe) pair**, not of the system: a deployment that only checks row counts would report near-zero detection on the same traffic. Any detection rate must be quoted with the probe depth that produced it.

4.5 Adaptive injection (E3)

guard	detection (in-sample)	FP	detection (out-of-sample)	FP
keyword rules (deployed)	33.0%	35.0%	31.0%	26.5%
intent rules (proposed)	**90.1%**	**2.5%**	**85.7%**	**3.5%**

The proposed guard adds Bosnian intent verbs with objects, Cyrillic homoglyph folding and an educational exemption (“how do I use…”). It generalizes with a 4.4 pp drop, but does not reach 95%: keyword or intent matching on text is structurally insufficient.

That 4.4 pp figure is an in-family split. A second generator, written independently with different strategies (polite/indirect, hypothetical, role-play, English, German, multi-step, authority, urgency, obfuscation, embedded-in-context; 372 attacks and 48 benign controls), cuts detection to **46.8% [41.8; 51.8]** with **6/48 false blocks**: German 0/36, English 33.3%, obfuscated 16.7%. The false blocks are all “mention vs intent” (quoted commands, a question about `chmod 777`, a file named `rm_rf_test.log`). The honest out-of-distribution drop is ~43 pp, which is why the action-level gate, not the text guard, carries the decision.

4.6 Action gate (RiskGate)

model	destructive calls on risky prompts	blocked
base	1	1 (100%)
g03a	7	7 (100%)
bal3	52	52 (100%)
bal5	23	23 (100%)
bal6	36	36 (100%)
total	**119**	**119 (100%)**

Legitimate traffic: 108/108 read-only calls allowed (**0 false blocks**); 92 explicitly requested destructive calls correctly escalated to confirmation. Note the spread in model behaviour: one fine-tune proposed 52 destructive calls in 64 risky prompts.

4.6b Attacking the gate itself

The numbers above measure what models happened to emit. We also attacked the gate directly with 1,000 destructive call variants (name obfuscation including Cyrillic homoglyphs and zero-width characters, shell destruction routed through `terminal`/`exec`, path attacks, SQL) and 500 benign read-only calls; nothing was executed.

gate	detection	false blocks
v1: exact tool-name list	**52.6%** [49.5; 55.7]	0/500
v2: normalisation + command/ path/SQL policy + fail-closed	**100%** [99.6; 100]	0/500

v1 detected 0/182 spelling variants, 0/36 SQL destructions and 85/288 shell destructions (`terminal {"command": "shutdown -h now"}` was allowed). v2 normalises names (NFKD, homoglyph folding, zero-width removal, separators, plurals), classifies destructive *intent* families, applies command/path/SQL policies to the arguments, and returns `confirm` for unknown tools. The stress set is ours and was written with knowledge of v1, so 100% is not an independent estimate; exotic obfuscations (base64 payloads, `$IFS`, bidi overrides) remain untested. The lesson is architectural: a name list is not a policy.

5. Discussion

The results separate two problems that are often merged. *Capability* (tool selection, retrieval, arithmetic) improves with data and size unevenly, and exact arithmetic remains unreliable at 5.7–37%. *Truthfulness* does not improve with fine-tuning at all in our setting; it degrades. The only mechanism in this campaign with a 100% record is a deterministic one: a rule engine over invoice fields (E6) and an action classifier over tool calls (RiskGate). A text-level guard sits in between and saturates around 86–90%.

This has a direct architectural consequence: the model should propose actions, and a separate deterministic component should decide execution and verify outcomes. Model-side “honesty training” is not a control.

6. Reproducibility

- Deterministic artifacts regenerate bit-identically (`e7_reproduce.py`): dataset SHA-256 `49c2f8ac…`, E6 gate `recall 1.0 / FP 0.0`, E3 guard counts `0.9015 / 0.025`.
- Template-level statistics: `benchmark/cluster_robust.py` → `benchmark/cluster_stats_2026-11-10.json` (24 clusters, deff 4.7–21.9).
- Human evaluation kit (200 items, 100 fictitious + 100 real, rubric and κ protocol): `benchmark/human_eval/`.
- Model outputs are **not** claimed to be bit-identical; `e2` documents 40/700 (TP=4) and 31/700 (TP=1) differing outputs even under strictly serial decoding.
- Artifacts: `benchmark/{gen_v2,run_v2,score_suite,e3_injection,e3_guard_v2,e6_gate,riskgate_validate,holdout_overlap,e7_reproduce}.py`, `docs/RESULTS_*.md`, `docs/SUMMARY_v1.5_2026-11-10.md`; raw runs in LUMI-O `gama-artifacts/`.
- Archived evidence pack v1.5.0: DOI `10.5281/zenodo.24118913` (bundle sha256 `344948a2…1897`).
- Private holdout (60 items) stays private; leakage is checked against every dev set (`holdout_overlap.py`), and one leaked item found this way was removed.

7. Limitations

- 12 prompt templates at N=10,000 (≈833 repetitions); it is not 10,000 distinct tasks.
- Bench v2 has 24 templates: effective N is 32–149 depending on the model, so the confidence intervals are wide and the per-template results (§4.2b) are the honest unit.
- Human evaluation is prepared as a 200-item kit with a κ protocol but has not yet been run with independent annotators.
- No human evaluation; all metrics are automatic and therefore narrow.
- Honesty items concern fictional entities: this is a lower bound on “do not invent”, not evidence of factual accuracy on real data. The real-entity set (§4.3b) uses Wikidata as reference, which may itself contain errors; its control condition has three seeds (fabrication 21.6–22.0%, 17.0–18.0%, 11.4–13.0%).
- The action gate’s allowlist is explicit; unknown tools currently default to `allow`, which should become fail-closed.

- Mistral-family models are excluded (serving defects), so the model sample is not representative of all open weights.